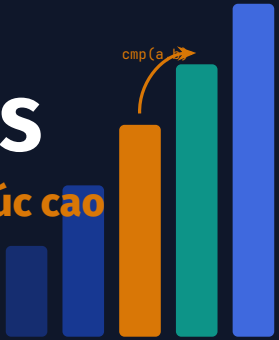
KHÓA HỌC C++ NÂNG CAO

COMPETITIVE PROGRAMMING

SORTING ALGORITHMS

Kỹ thuật Xử lý & Sắp xếp Dữ liệu Cấu trúc cao



Làm chủ `std::sort`, cơ chế **Custom Comparator** (`cmp`), cấu trúc `std::pair`, thiết kế `struct` đa tiêu chí và phân tích độ phức tạp trong lập trình thi đấu.

Tài liệu bài giảng chuẩn hóa · C++ Standard Template Library (`<algorithm>`)

Nội dung bài học

Mở đầu: Vai trò của Sắp xếp trong CP

- Tầm quan trọng của bước tiền xử lý dữ liệu.
- Sử dụng `std::sort` cơ bản và giới hạn của `greater<T>()`.
- Bài toán thực tế đặt ra và 3 mảnh ghép cốt lõi.

1. Custom Comparator (Hàm Compare)

- Bản chất câu hỏi định hướng của `cmp(A, B)`.
- Quy tắc **Strict Weak Ordering** (`a == b` trả về `false`).
- Giải quyết bài toán sắp xếp Lẻ trước – Chẵn sau.

2. Cấu trúc dữ liệu `std::pair`

- Quản lý cặp thuộc tính đi liền nhau (Tọa độ, Trọng số, Thời gian).
- Cơ chế sắp xếp từ điển (*Lexicographical Order*) mặc định.

3. Cấu trúc `struct` & Mở rộng chuyên sâu

- Thiết kế `struct` đa thuộc tính & tối ưu tham chiếu `const &`.
- Minh họa trực quan và Trace luồng dữ liệu trên hệ trục *Oxy*.
- **Mở rộng** Lambda, `stable_sort` & Độ phức tạp.

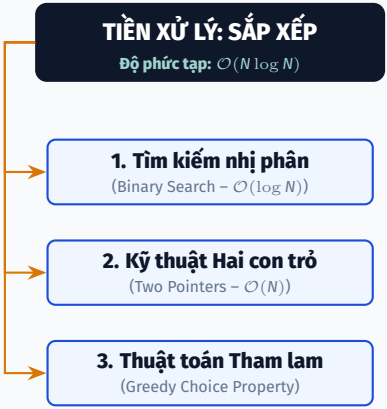
Tầm quan trọng của Sắp xếp trong CP

Thống kê thực tế trong Competitive Programming

Sắp xếp là một trong những nền tảng quan trọng nhất trong lập trình thi đấu (CP). Khoảng **60% – 70%** các bài toán cần tối ưu dữ liệu đều sử dụng sắp xếp như một **bước tiền xử lý** (preprocessing) bắt buộc.

Tại sao phải sắp xếp trước khi xử lý?

- Đưa dữ liệu hỗn loạn về trạng thái có **tính đơn điệu** (Monotonicity).
- Giảm độ phức tạp từ duyệt vét cạn $\mathcal{O}(N^2)$ xuống $\mathcal{O}(N \log N)$.
- Mở khóa 3 kỹ thuật thuật toán trụ cột ở sơ đồ bên cạnh.



Sử dụng `std::sort` cơ bản

Thông thường, ta sử dụng hàm `std::sort(first, last)` trong thư viện `<algorithm>` với độ phức tạp $\mathcal{O}(N \log N)$.

1. Sắp xếp tăng dần (Mặc định)

```cpp
int a[5] = {5, 2, 3, 9, 7};
std::sort(a, a + 5);
// Mảng a sau khi sort: {2, 3, 5, 7, 9}
```



2. Sắp xếp giảm dần với `std::greater<T>()`

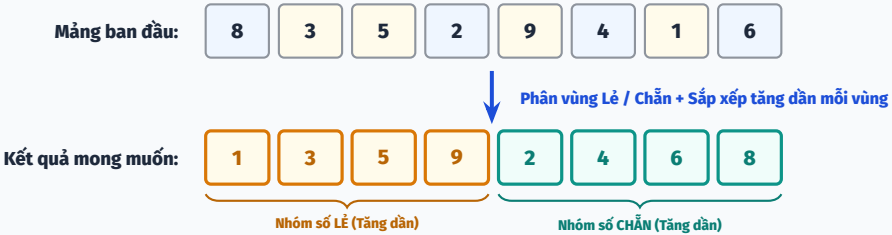
```cpp
int a[5] = {5, 2, 3, 9, 7};
std::sort(a, a + 5, std::greater<int>());
// Mảng a sau khi sort: {9, 7, 5, 3, 2}
```



Vấn đề thực tế

Bài toán đặt ra

Cho mảng số nguyên A. Hãy sắp xếp mảng sao cho **tất cả các số lẻ đứng trước và xếp tăng dần**, tiếp theo là **tất cả các số chẵn đứng sau và cũng xếp tăng dần**.



- Lúc này, cả `std::sort` mặc định lẫn `std::greater<int>()` đều **hoàn toàn bất lực**!
- Đó là lý do bạn bắt buộc phải làm chủ kỹ thuật **Custom Comparator (Hàm Compare)**.

Mục tiêu bài học

MẢNH GHÉP 01

Custom Comparator

(Hàm Compare tùy biến)

Hiểu rõ bản chất cơ học (mechanics) của hàm so sánh để **tự định nghĩa mọi quy tắc sắp xếp phức tạp** theo yêu cầu đề bài.

MẢNH GHÉP 02

Cấu trúc `std::pair`

(Cặp dữ liệu liên kết)

Khai thác cơ chế sắp xếp **thứ tự từ điển mặc định** của `pair` để xử lý cực nhanh các bài toán tọa độ, đoạn thẳng, trọng số đồ thị.

MẢNH GHÉP 03

Cấu trúc `struct`

(Kiểu dữ liệu đa thuộc tính)

Thiết kế kiểu dữ liệu tùy biến cho các bài toán mô phỏng thực tế nhiều tiêu chí và **tối ưu hóa bộ nhớ** với tham chiếu hằng.

PHẦN 01

Custom Comparator (Hàm Compare)

Hiểu rõ cơ chế truyền hàm so sánh vào `std::sort`, quy tắc Strict Weak Ordering và giải quyết bài toán phân vùng Chẵn – Lẻ.

Bản chất của hàm Compare

Định nghĩa hàm Compare (`cmp`)

- Hàm compare (thường đặt tên là `cmp`) là một hàm nhận vào 2 phần tử A, B và trả về kiểu dữ liệu `bool`.
- Khi truyền `cmp` vào `std::sort`, bạn đang định nghĩa quy tắc trả lời câu hỏi:

"Khi nào thì phần tử A được phép đứng TRƯỚC phần tử B?"

Trường hợp `cmp(A, B)` trả về `true`

A có mức độ ưu tiên cao hơn B.
$\Longrightarrow$ A **được xếp đứng trước** B trong mảng kết quả: $[A, \ldots, B]$.

Trường hợp `cmp(A, B)` trả về `false`

A **không** có độ ưu tiên cao hơn B.
$\Longrightarrow$ B sẽ đứng trước A (hoặc giữ nguyên nếu cùng mức ưu tiên).

Ví dụ đầu tiên: Sắp xếp giảm dần

Mục tiêu: Sắp xếp mảng a gồm n phần tử theo thứ tự giảm dần.

```
bool cmp(int a, int b) {
    return a > b;
}

// Nếu mảng đánh chỉ số từ 1 đến n (1-indexed):
sort(a + 1, a + n + 1, cmp);

// Nếu mảng đánh chỉ số từ 0 đến n-1 (0-indexed):
sort(a, a + n, cmp);
```

Giải mã ý nghĩa toán học

$$\texttt{cmp}(a, b) == \texttt{true}$$

$$\Updownarrow$$

a **được phép đứng trước** b

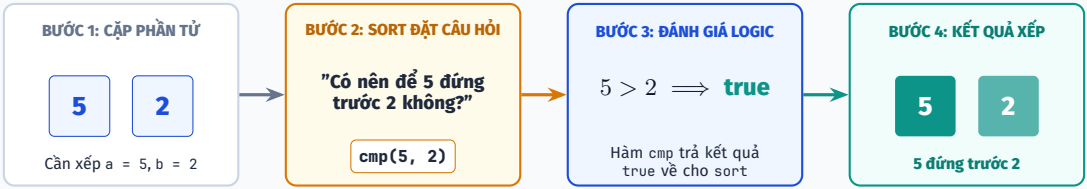
$$\Updownarrow$$

$$\mathbf{a > b}$$

Lưu ý: Khi truyền `cmp` vào `sort`, ta chỉ truyền **tên hàm** `cmp` (con trỏ hàm), không viết dấu ngoặc tròn `cmp()`.

Minh họa cách `std::sort` gọi hàm Compare

Giả sử trong quá trình sắp xếp giảm dần, thuật toán cần quyết định thứ tự giữa hai số **5** và **2**:



Nhận xét cốt lõi

Bạn **không cần** tự viết vòng lặp đổi chỗ (`swap`). Nhiệm vụ duy nhất của lập trình viên là viết hàm `cmp(a, b)` chính xác; toàn bộ quá trình phân hoạch và đổi chỗ đã được `std::sort` thực hiện tự động.

Lưu ý quan trọng khi viết Compare (1/2)

Quy tắc bắt buộc cần nhớ

Nếu hai phần tử **bằng nhau** ($a = b$), hàm compare **BẮT BUỘC phải trả về** `false`:

$$\forall x, \quad \texttt{cmp}(x, x) == \texttt{false} \quad \text{(Tính chất phản xạ nghiêm ngặt – Irreflexivity)}$$

Ví dụ ĐÚNG (Sử dụng dấu > hoặc <):

```
1 bool cmp(int a, int b) {
2     return a > b; // Chính xác!
3 }
```

Kiểm tra tại biên $a = b = 5$

- Gọi `cmp(5, 5)`: Biểu thức $5 > 5$ trả về `false`.
- Ý nghĩa: Số 5 thứ nhất **không nhất thiết** phải vượt lên trước số 5 thứ hai $\Longrightarrow$ Thuật toán dừng so sánh cặp này an toàn.

Lưu ý quan trọng khi viết Compare (2/2)

Ví dụ sai:

```
bool cmp(int a, int b) {
    return a >= b; // SAI NGHIÊM TRỌNG!
}
```

Mâu thuẫn logic khi $a = b = 5$

- `cmp(5_trái, 5_phải)` → `true` (5 trái đòi đứng trước).
- `cmp(5_phải, 5_trái)` → `true` (5 phải cũng đòi đứng trước).

Hậu quả vi phạm

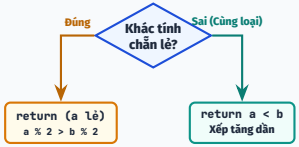
Khi vi phạm Strict Weak Ordering, hàm `std::sort()` mất điểm neo dừng con trỏ phân hoạch (QuickSort Partition), dẫn đến **Undefined Behavior**:

1. **Runtime Error (RE / Segmentation Fault):** Con trỏ dịch chuyển vượt quá biên mảng khi tìm chốt (pivot).
2. **Time Limit Exceeded (TLE):** Rơi vào vòng lặp hoán đổi vô tận giữa hai phần tử bằng nhau.
3. **Wrong Answer (WA):** Kết quả mảng đầu ra bị xáo trộn không xác định.

Giải quyết bài toán ban đầu: Lẻ trước, Chẵn sau

Yêu cầu: Sắp xếp các số **lẻ đứng trước (tăng dần)**, các số **chẵn đứng sau (tăng dần)**.
Ta cần xác định: **Khi nào thì** `cmp`$(a, b) =$ `true` (a được đứng trước b)?

Tính chất a	Tính chất b	`cmp(a, b)`	Giải thích quy tắc
Lẻ	Chẵn	true	Khác tính chẵn lẻ, a lẻ được ưu tiên đứng trước b.
Chẵn	Lẻ	false	a chẵn phải nhường vị trí đứng trước cho b lẻ.
Lẻ	Lẻ	`a < b`	Cùng lẻ → số nhỏ hơn đứng trước (tăng dần).
Chẵn	Chẵn	`a < b`	Cùng chẵn → số nhỏ hơn đứng trước (tăng dần).



Cài đặt hàm Compare: Lẻ trước, Chẵn sau

Cài đặt theo tài liệu gốc (Dành cho số nguyên dương):

```
bool cmp(int a, int b) {
    if (a % 2 != b % 2)
        return a % 2 > b % 2;
    return a < b;
}
```

Ý tưởng cốt lõi:

- **Khác tính chẵn lẻ** (`a % 2 != b % 2`): Số lẻ có số dư là 1, số chẵn có số dư là 0. Biểu thức `a % 2 > b % 2` giúp số lẻ luôn được ưu tiên.
- **Cùng tính chẵn lẻ**: Xét dòng `return a < b;` để sắp xếp tăng dần.

Mở rộng CP **Bẫy số nguyên âm với `% 2`**

Trong C++, nếu $a = -3$ (số âm lẻ), phép toán `-3 % 2` trả về `-1` (nhỏ hơn `0`), khiến biểu thức `a % 2 > b % 2` bị sai!

Cách viết an toàn tuyệt đối cho mọi số nguyên $\mathbb{Z}$:

```
bool cmpSafe(int a, int b) {
    // Dùng phép AND bit: lẻ = 1, chẵn = 0
    if ((a & 1) != (b & 1))
        return (a & 1) > (b & 1);
    return a < b;
}
```

PHẦN 02

Cấu trúc dữ liệu `std::pair`

Khai thác vũ khí sắp xếp theo thứ tự từ điển (Lexicographical Order) mặc định để tăng tốc độ code trong thi đấu.

Cấu trúc dữ liệu `std::pair`

Trong CP, ta thường xuyên quản lý các đối tượng có **2 thuộc tính đi liền nhau**:

- **Hình học:** Tọa độ điểm (X, Y) trên mặt phẳng.
- **Đồ thị (Dijkstra / Prim):** Cặp (Trọng số, Chỉ số đỉnh) trong hàng đợi ưu tiên.
- **Tham lam (Scheduling):** Thời gian (Bắt đầu, Kết thúc) của một công việc.

Cú pháp khai báo & truy cập cơ bản (`<utility>`):

```cpp
pair<string, int> student;
student.first = "Duc Cong";
student.second = 2025;

// Khởi tạo nhanh (C++11 trở lên):
pair<int, int> point = {10, 20};
```

Giải phẫu bộ nhớ `pair<string, int>`



Cơ chế `sort` mặc định với `std::pair`

Mặc định, `std::sort` với `pair` tự động so sánh theo quy tắc 2 bước:

1. **Tiêu chí 1:** Ưu tiên so sánh thuộc tính `.first` trước (tăng dần).
2. **Tiêu chí 2:** Nếu `.first` hoàn toàn bằng nhau, hệ thống tự động xét tiếp `.second` (tăng dần).

Ví dụ: Sắp xếp mốc thời gian (Giờ, Phút) tăng dần

```cpp
vector<pair<int, int>> times = {
    {14, 30}, {9, 15}, {14, 15}, {9, 5}
};
sort(times.begin(), times.end());

for (auto t : times) {
    cout << t.first << ":" << t.second << " | ";
}
// Result: 9:5 | 9:15 | 14:15 | 14:30 |
```

Mẹo thực chiến CP

Hãy chủ động đặt **thuộc tính ưu tiên sắp xếp lên làm** `.first` để tận dụng ngay `sort()` mặc định, tiết kiệm thời gian viết hàm `cmp`.



PHẦN 03

Cấu trúc dữ liệu `struct`

Giải pháp chuyên nghiệp cho các bài toán quản lý đối tượng nhiều thuộc tính và sắp xếp đa tiêu chí (Multi-criteria Sorting).

Cấu trúc `struct`

Khi bài toán vượt quá 2 thuộc tính (VD: Thí sinh có *Mã số*, *Họ tên*, *Điểm Toán*, *Lý*, *Hóa*), việc lồng `pair` (`p.first.second.first`) sẽ tạo ra **sự phức tạp** $\Longrightarrow$ `struct` là giải pháp chuyên nghiệp.

Khởi tạo Struct cơ bản:

```
struct Point {
    int x, y;
}; // Đừng quên dấu ; ở cuối struct
```

Nguyên tắc khi sắp xếp `struct` :

- `struct` **không có** cơ chế so sánh tự động mặc định như `pair` .
- Bắt buộc phải định nghĩa quy tắc so sánh qua hàm `cmp` (hoặc nạp chồng `operator<`).

Mẹo tối ưu hiệu năng: Tham chiếu hằng `const &`

Luôn truyền tham số vào hàm `cmp` dưới dạng `const StructName&` để:

1. **Tránh copy object:** Chỉ truyền địa chỉ ($\mathcal{O}(1)$) thay vì sao chép toàn bộ dữ liệu của `struct` ở mỗi phép so sánh.
2. **An toàn dữ liệu:** Từ khóa `const` đảm bảo hàm so sánh không vô tình làm thay đổi giá trị phần tử.

Bài toán minh họa `struct`

Yêu cầu bài toán

Cho danh sách các điểm trên mặt phẳng tọa độ *Oxy* (mỗi điểm có `id`, hoành độ `x`, tung độ `y`).

Hãy sắp xếp các điểm **tăng dần theo hoành độ** *X*. Nếu *X* bằng nhau, điểm nào có **tung độ** *Y* **nhỏ hơn sẽ đứng trước** (tăng dần theo *Y*).

```cpp
struct Point {
    int id; // ID theo dõi vị trí trong mảng ban đầu
    int x, y;
};

bool comparePoints(const Point& a, const Point& b) {
    if (a.x != b.x) {
        return a.x < b.x; // Tiêu chí 1: Tăng dần theo hoành độ X
    }
    return a.y < b.y;     // Tiêu chí 2: Nếu X bằng nhau, tăng dần theo tung độ Y
}
```

Ứng dụng `struct` trong hàm `main()`

```cpp
int main() {
    vector<Point> plist = {
        {1, 5, 10}, // P1: id = 1, tọa độ (5, 10)
        {2, 2, 8},  // P2: id = 2, tọa độ (2, 8)
        {3, 5, 3},  // P3: id = 3, tọa độ (5, 3)
        {4, 2, 1},  // P4: id = 4, tọa độ (2, 1)
        {5, 9, 4}   // P5: id = 5, tọa độ (9, 4)
    };

    // Sắp xếp vector<Point> với hàm comparePoints đã viết
    sort(plist.begin(), plist.end(), comparePoints);

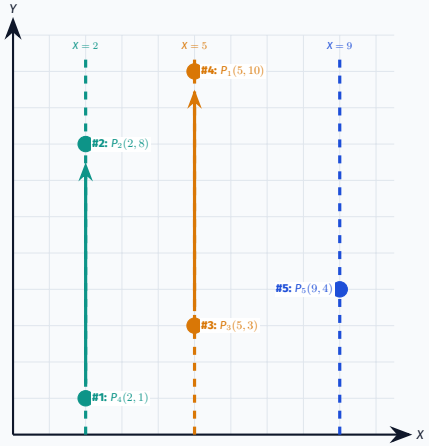
    cout << "ID\tX\tY" << endl;
    for (const auto& p : plist) {
        cout << p.id << "\t" << p.x << "\t" << p.y << endl;
    }
    return 0;
}
```

Giải thích luồng xử lý dữ liệu (Trace)

- **Nhóm có $X = 2$:** Gồm $P_2(2, 8)$ và $P_4(2, 1)$.
  Vì hoành độ *X* bằng nhau, xét tiếp tung độ *Y*: do $1 < 8$ nên P_4 **đứng trước** P_2.
- **Nhóm có $X = 5$:** Gồm $P_1(5, 10)$ và $P_3(5, 3)$.
  Tương tự, hoành độ bằng nhau nên xét tung độ *Y*: do $3 < 10$ nên P_3 **đứng trước** P_1.
- **Nhóm có $X = 9$:** Điểm $P_5(9, 4)$ có hoành độ lớn nhất toàn cục nên xếp ở **cuối danh sách**.

Nhận xét tổng quan

Cột *X* được sắp tăng dần $(2 \to 2 \to 5 \to 5 \to 9)$. Tại các **điểm gãy** nơi *X* trùng nhau, cột *Y* đóng vai trò quyết định thứ tự.



Kết quả sau khi sắp xếp

Trước khi sắp xếp (Input)

ID (`p.id`)	Hoành độ X	Tung độ Y
1	5	10
2	2	8
3	5	3
4	2	1
5	9	4



Sau khi sắp xếp (Output chuẩn)

ID (`p.id`)	Hoành độ X	Tung độ Y
4	**2**	**1**
2	**2**	**8**
3	**5**	**3**
1	**5**	**10**
5	**9**	**4**

Quan sát sự thay đổi của cột `ID`

Nhờ lưu thuộc tính `id` ban đầu trong `struct Point`, sau khi sắp xếp ta vẫn truy vết được vị trí gốc của từng điểm ($4 \to 2 \to 3 \to 1 \to 5$). Đây là kỹ thuật cốt lõi khi đề bài yêu cầu **in ra chỉ số ban đầu** sau khi tối ưu.

PHẦN 04

[Mở rộng] Kỹ thuật C++ Hiện đại & Độ phức tạp

Nâng cấp kỹ năng với Lambda Expression, nạp chồng toán tử `operator<`, tính ổn định (`std::stable_sort`) và bảng phân tích độ phức tạp toàn diện.

Mở rộng Cú pháp C++ hiện đại cho Struct

Bên cạnh cách viết hàm `comparePoints` độc lập, các kỳ thi CP hiện đại thường dùng 2 kỹ thuật gọn nhẹ sau:

Cách 1: Nạp chồng `operator<` trong `struct`

```
struct Point {
    int id, x, y;
    bool operator<(const Point& other) const {
        if (x != other.x)
            return x < other.x;
        return y < other.y;
    }
};
// Gọi sort trực tiếp như pair:
sort(plist.begin(), plist.end());
```

Cách 2: Biểu thức Lambda ngay trong `sort`

```
sort(plist.begin(), plist.end(),
    [](const Point& a, const Point& b) {
        if (a.x != b.x)
            return a.x < b.x;
        return a.y < b.y;
    }
);
// Mẹo: dùng std::tie(a.x, a.y) < std::tie(b.x, b.y)
// (<tuple>) để viết gọn trong 1 dòng!
```

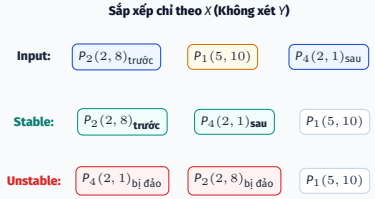
Mở rộng Tính ổn định (Stability) trong Sắp xếp

Định nghĩa Tính ổn định (Stability)

Một thuật toán sắp xếp được gọi là **ổn định (Stable)** nếu hai phần tử có **cùng khóa so sánh** vẫn giữ nguyên thứ tự xuất hiện tương đối của chúng như trong mảng ban đầu.

- `std::sort` **(Không ổn định – Unstable):** Cài đặt bằng **Introsort** (lai giữa QuickSort, HeapSort và InsertionSort). Các phần tử bằng nhau có thể bị đảo vị trí tương đối.
- `std::stable_sort` **(Ổn định – Stable):** Cài đặt dựa trên **MergeSort**. Giữ nguyên thứ tự ban đầu của các phần tử có khóa bằng nhau:

```
1 std::stable_sort(v.begin(), v.end(), cmp);
```



Mở rộng Phân tích Độ phức tạp & Đặc tính Thuật toán

Thuật toán / Hàm STL	Best Case	Average Case	Worst Case	Space	Stable	In-place
`std::sort` (Introsort)	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(\log N)$	**Không**	**Có**
`std::stable_sort`	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	**Có**	**Không**
QuickSort (Cổ điển)	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N^2)$	$\mathcal{O}(\log N)$	Không	Có
MergeSort	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	Có	Không
HeapSort	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(N \log N)$	$\mathcal{O}(1)$	Không	Có
Insertion Sort	$\mathcal{O}(N)$	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	$\mathcal{O}(1)$	Có	Có
Counting Sort	$\mathcal{O}(N + K)$	$\mathcal{O}(N + K)$	$\mathcal{O}(N + K)$	$\mathcal{O}(K)$	Có	Không

Khi nào sử dụng công cụ nào trong thực tế?

- **95% bài toán CP:** Dùng `std::sort` vì hằng số thời gian cực nhỏ, bộ nhớ $\mathcal{O}(\log N)$ và đảm bảo không bao giờ bị suy biến thành $\mathcal{O}(N^2)$.
- **Bài toán có 2 thuộc tính cần code nhanh:** Dùng `std::pair<T1, T2>` + `std::sort` mặc định.
- **Bài toán ≥ 3 thuộc tính hoặc quy tắc phức tạp:** Dùng `struct` + hàm `cmp(const T& a, const T& b)`.

Tổng kết bài học

Tóm tắt 3 trụ cột kiến thức

1. **Custom Comparator (`cmp`):** Trả lời câu hỏi *"Khi nào A đứng trước B?"*. Luôn đảm bảo $\texttt{cmp}(x, x) = \texttt{false}$.
2. **Cấu trúc `std::pair` :** Tự động sắp xếp theo thứ tự từ điển (`.first` trước, `.second` sau).
3. **Cấu trúc `struct` :** Quản lý đối tượng đa thuộc tính rõ ràng, kết hợp `const T&` để tối ưu tốc độ.

4 lỗi sai kinh điển cần kiểm tra

- × Dùng dấu `⩾` hoặc `⩽` trong hàm `cmp` (gây RE/TLE).
- × Quên dấu chấm phẩy `;` kết thúc khai báo `struct` .
- × Truyền tham trị `Point a` thay vì `const Point& a` khi `struct` lớn.
- × Dùng `a % 2 == 1` cho số nguyên có thể âm (nên dùng `(a & 1)` hoặc `a % 2 != 0`).

Chúc các bạn làm chủ kỹ thuật Sorting và bứt phá trong các kỳ thi Competitive Programming!